

EfficientCaps: A Lightweight Capsule Network for Robust Traffic Sign Recognition Under Real-World Degradation

Mansoureh Fahimi
Student ID: 2359226

`mansoureh.fahimi@bahcesehir.edu.tr`

Abstract

Traffic sign recognition in real-world driving conditions is much more difficult than standard benchmark results suggest. In real environments, traffic signs can be affected by **snow, rain, blur, low lighting, or poor image quality**. Most existing models are either tested only on clean datasets or trained directly on degraded images to handle these conditions.

This paper presents **EfficientCaps**, a lightweight Capsule Network trained only on clean GTSRB data. The main goal is to investigate whether a model trained on clean images can still recognize degraded real-world traffic signs without using degraded images during training.

To improve robustness, EfficientCaps is combined with an inference pipeline including **sign localization, adaptive image enhancement**, and **Test-Time Augmentation (TTA)**. A **class-weighted margin loss** is also introduced to improve recognition of difficult traffic sign classes.

The proposed model is compared with several re-implemented baseline models — **Cireşan *et al.*, CSNN+CBAM, EfficientNet-B0, and MobileNetV2** — under the same training and testing conditions.

EfficientCaps achieves **99.1%** accuracy on the standard GTSRB test set and correctly classifies **51.5%** of a curated set of 33 real-world degraded traffic sign images, with a **Macro F1 score of 96.5%** and no class falling below 80% F1.

The results suggest that capsule-based feature learning combined with adaptive preprocessing can help improve robustness to real-world image degradation **without requiring degraded training data**.

1. Introduction

Traffic sign recognition is an important task in autonomous driving and driver-assistance systems. A vehicle that incorrectly recognizes a stop sign or misses a speed limit sign under bad weather can make decisions that are genuinely dangerous. Although many models report very high accu-

racy on benchmark datasets, real-world driving conditions are much more challenging. Traffic signs may appear under **rain, snow, fog, night lighting, motion blur, or low image quality**, any of which can cause a well-trained model to fail.

Most existing studies address robustness by training models on degraded or synthetically corrupted images, or by using domain adaptation techniques. This work explores a different direction: whether a model trained only on clean traffic sign images can still generalize to degraded real-world conditions without using degraded data during training.

To investigate this, the work focuses on **Capsule Networks**. Unlike standard CNNs, capsule networks encode spatial and geometric relationships using vector-based representations, which may provide greater stability under viewpoint changes, partial occlusion, and image degradation.

However, traditional capsule networks carry high computational cost due to iterative routing. To address this, **EfficientCaps** is proposed — a lightweight capsule-based architecture using a depthwise-separable feature extractor and a simplified single-step routing mechanism, designed to reduce computational cost while preserving the representational benefits of capsule networks.

Contributions.

- A **lightweight capsule-based architecture** (EfficientCaps) for traffic sign recognition, trained from scratch on GTSRB without any pre-training, achieving **99.1%** clean accuracy.
- A **class-weighted training strategy** combining a weighted margin loss, balanced sampling, and stronger augmentation for difficult classes, without synthetic corruption training.
- An **adaptive inference pipeline** with sign localization, condition-aware image enhancement, and Test-Time Augmentation (TTA) to improve performance on degraded real-world images.
- A **simplified routing mechanism** that reduces the computational cost of traditional capsule networks while maintaining equivariant feature representations.

The experimental results show that EfficientCaps performs better than the compared baseline models on real-world degraded traffic sign images, even though it was trained only on clean data. The results suggest that capsule-based representations combined with adaptive preprocessing can be a useful approach for improving robustness under difficult real-world conditions.

2. Related Work

2.1. Traffic Sign Recognition

Early traffic sign recognition methods relied on hand-crafted features and traditional classifiers. The field shifted significantly after Cireřan *et al.* [7] demonstrated that deep CNNs with strong augmentation could achieve high accuracy on the GTSRB dataset. Subsequent work applied Spatial Transformer Networks [4] for implicit alignment, and attention-based models such as CSNN+CBAM [5] to focus on informative sign regions.

Most of these models are evaluated on clean benchmark images. In real driving conditions, however, signs can appear under **snow, rain, blur, low lighting, or degraded image quality**, which makes recognition considerably more difficult and is not captured by standard benchmarks.

2.2. Capsule Networks

Capsule Networks were introduced by Sabour *et al.* [6] as an alternative to standard CNNs. Rather than using scalar activations, capsule networks encode spatial relationships through **vector representations**, which may provide better stability under geometric transformations and partial degradation compared to standard pooling-based approaches.

One limitation of the original design is its **high computational cost** due to iterative dynamic routing. In this work, a simplified single-step routing mechanism and a lightweight depthwise-separable backbone are used to reduce this cost.

2.3. Robustness to Image Degradation

Robustness to image corruptions has been widely studied using synthetic benchmarks [1], with approaches such as AugMix [2] and test-time adaptation [8]. For traffic sign recognition specifically, training on synthetically corrupted images is the most common strategy.

This work takes a different approach: the model is trained **only on clean GTSRB data**, and robustness is addressed through capsule-based representations, targeted training strategies, and an adaptive inference pipeline applied at test time.

3. Methods

3.1. Model Architecture

EfficientCaps has four components. Each one is designed to be both efficient and well-suited for traffic sign recognition.

Backbone. The backbone takes a $3 \times 48 \times 48$ input and extracts a $256 \times 6 \times 6$ feature map:

$$\mathbf{F} = \text{DSC}_{128 \rightarrow 256}(\text{DSC}_{64 \rightarrow 128}(\text{Conv}_{3 \rightarrow 64}(\mathbf{x}))), \quad (1)$$

where $\text{Conv}_{3 \rightarrow 64}$ is a 5×5 convolution with batch normalisation and ReLU, and each DSC block is a depthwise-separable convolution [3] with stride 2. Depthwise-separable convolutions are chosen to keep the parameter count low.

Primary Capsule Layer. 32 parallel DSC heads are applied to $\mathbf{F}$, each producing an 8-dimensional output. After reshaping, 288 primary capsules of dimension 8 are obtained. These are squashed to unit length using the squash function [6]:

$$\text{squash}(\mathbf{v}) = \frac{\|\mathbf{v}\|^2}{1 + \|\mathbf{v}\|^2} \cdot \frac{\mathbf{v}}{\|\mathbf{v}\|}. \quad (2)$$

Efficient Routing Layer. Instead of iterative dynamic routing, a single learned routing step is used. A weight tensor $\mathbf{W} \in \mathbb{R}^{P \times 43 \times 8 \times 16}$ produces prediction vectors, and a learned bias $\mathbf{b}$ controls the routing:

$$\mathbf{v}_j = \text{squash}\left(\sum_i c_{ij} \mathbf{u}_i \mathbf{W}_{ij}\right), \quad c_{ij} = \text{softmax}(\mathbf{b})_{ij}. \quad (3)$$

The classification logit for class j is $\ell_j = \|\mathbf{v}_j\|_2$. The difference between the top-1 and top-2 norms gives a useful *capsule agreement score* used as a confidence proxy at inference time.

Reconstruction Decoder. During training only, a three-layer MLP decodes the winning class capsule back to the input image. This reconstruction branch forces the class capsules to retain spatially precise information and acts as a regulariser.

3.2. Training Strategy

Weighted Margin Loss. Some classes — especially “Beware of ice/snow” (class 30) and “End of no passing” (class 41) — were consistently misclassified in early experiments. This is addressed by a class-weighted margin loss that assigns a higher penalty to errors on ten hard classes, with $m^+ = 0.9$, $m^- = 0.1$, $\alpha = 0.5$:

$$\mathcal{L}_{\text{margin}} = \sum_{k=1}^{43} \lambda_k [T_k \max(0, m^+ - \|\mathbf{v}_k\|)^2 + \alpha(1 - T_k) \max(0, \|\mathbf{v}_k\| - m^-)^2]. \quad (4)$$

The penalty weights λ_k range from 1.5 to 3.0 for the ten hard classes (see Table 1), based on per-class F1 analysis on

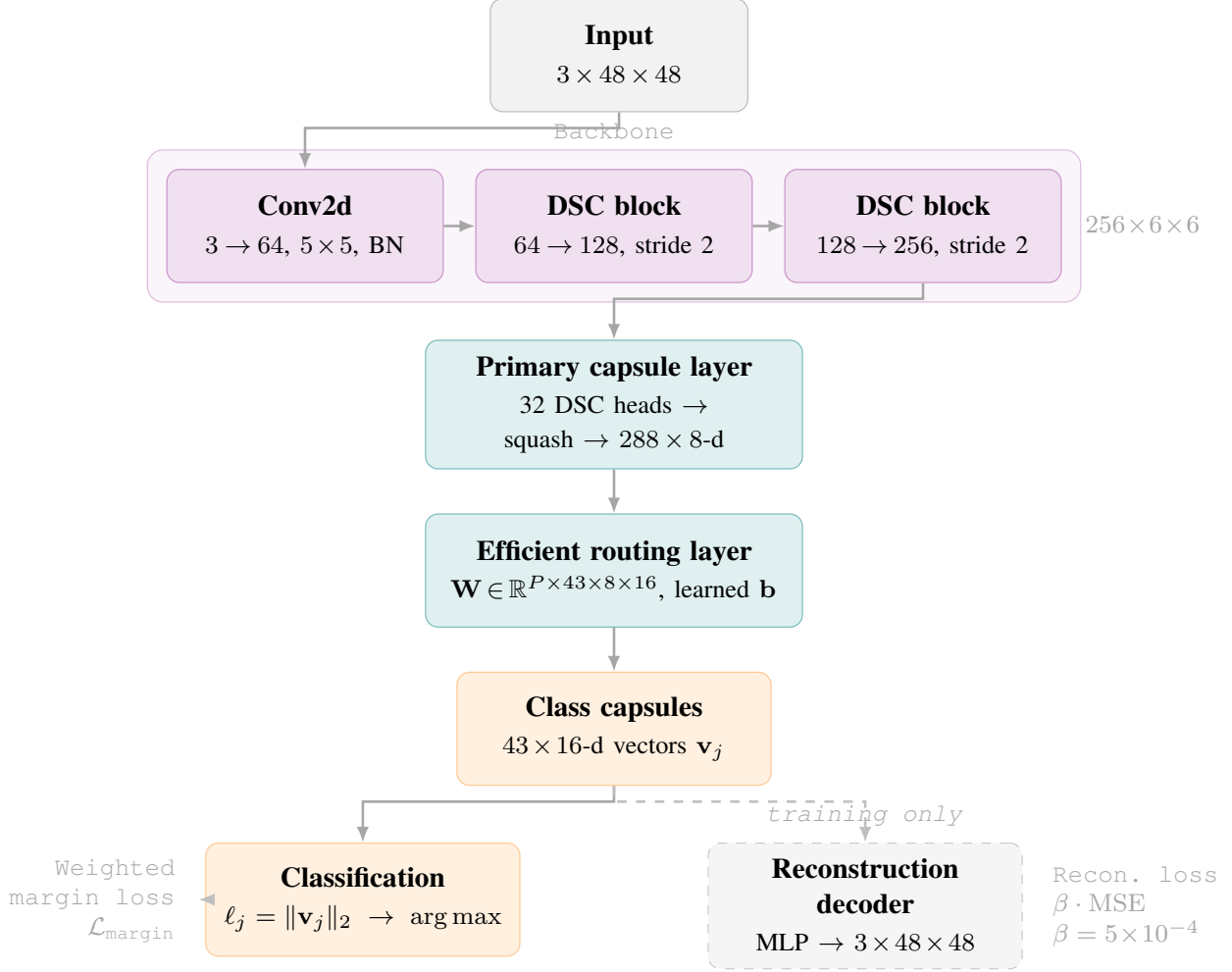


Figure 1. **EfficientCaps architecture.** A depthwise-separable backbone (purple) extracts $256 \times 6 \times 6$ feature maps. The Primary Capsule Layer applies 32 parallel DSC heads, squashes the outputs, and produces 288 primary capsules of dimension 8 (teal). The Efficient Routing Layer transforms them into 43 class capsules of dimension 16 via a single learned routing step. Classification logits are ℓ_2 norms of class capsules (orange). A Reconstruction Decoder (dashed) provides regularisation during training only.

the validation set. The total loss adds a small reconstruction term: $\mathcal{L} = \mathcal{L}_{\text{margin}} + 0.0005 \cdot \text{MSE}(\hat{\mathbf{x}}, \mathbf{x})$.

Table 1. Hard classes with elevated loss weights.

Class	Name	λ_k
30	Beware of ice/snow	3.0
41	End of no passing	3.0
23	Slippery road	2.5
19	Dangerous curve (L)	2.0
20	Dangerous curve (R)	2.0
21	Double curve	2.0
27	Pedestrians	2.0
22	Bumpy road	1.5
18	General caution	1.5
0	Speed limit (20 km/h)	1.5

Hard-Class Augmentation. For these ten classes, a stronger augmentation branch is applied with probability 0.70 per sample, using wider affine ranges, stronger colour jitter, and random perspective distortion.

Balanced Sampler. A `WeightedRandomSampler` with inverse-frequency weights plus extra boosts for the hard classes ($2 \times -3 \times$) is used so these classes appear more often during training without discarding other samples.

3.3. Inference Pipeline

At test time, an inference pipeline is applied before the final prediction. The goal of this pipeline is to improve recognition performance on degraded real-world traffic sign images. Fig. 2 gives a high-level overview of the full flow, while Fig. 3 details each step.

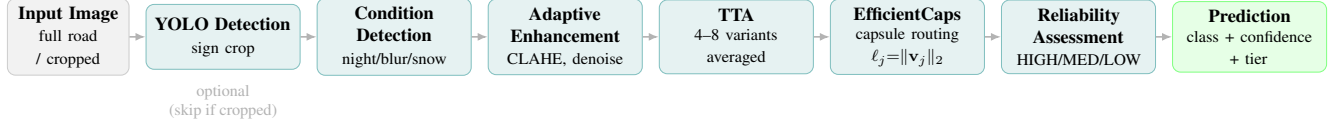


Figure 2. **EfficientCaps inference pipeline (overview)**. A full road image passes through optional sign detection (YOLOv8n), condition-aware enhancement, test-time augmentation, capsule-based classification, and a final reliability assessment that assigns a confidence tier to each prediction.

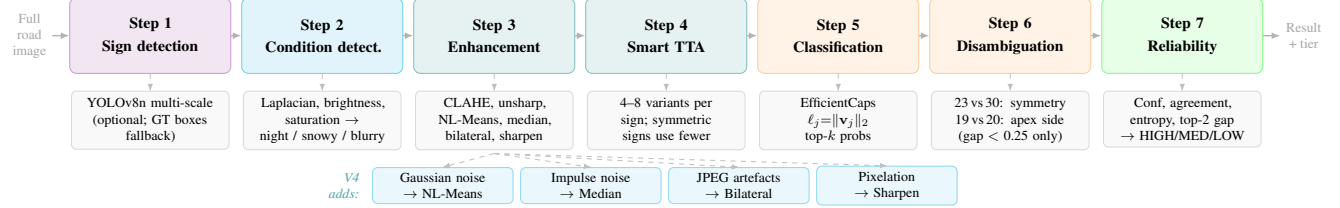


Figure 3. **EfficientCaps inference pipeline (detailed, V3/V4)**. Each of the seven steps is applied sequentially to a test image. Step 1 (sign detection) is optional; the remaining steps operate on the cropped sign region. EfficientCaps-V4 extends Step 3 with four additional adaptive filters (bottom row) triggered by runtime degradation detection. Disambiguation (Step 6) is only active when the top-1/top-2 confidence gap is below 0.25.

Step 1 — Sign Localisation. For full road images, YOLOv8n is used to detect and crop the traffic sign region before classification. For already cropped images, this step is skipped.

Step 2 — Condition Detection. After cropping, several image properties such as **brightness, sharpness, and saturation** are measured to estimate the image condition. Based on these values, the image is classified into conditions such as *night*, *snowy*, *blurry*, or *normal*.

Step 3 — Adaptive Enhancement. Different enhancement methods are applied depending on the detected condition. Dark images use **CLAHE** and brightness enhancement, blurry images use sharpening, and snowy or faded images use contrast enhancement. Additional filters are also applied for different noise types and compression artefacts.

Step 4 — Test-Time Augmentation (TTA). Multiple augmented versions of the image are passed through the model during testing, and the prediction probabilities are averaged. This helps improve stability and reduce prediction errors.

Step 5 — EfficientCaps Classification. The enhanced image is then passed into the EfficientCaps model, which produces the final class probabilities.

Step 6 — Disambiguation. Some traffic sign classes are visually very similar. To reduce confusion between these classes, additional **geometric checks** are applied in difficult cases such as slippery road vs. ice/snow signs (classes 23 and 30) and left vs. right dangerous curve signs (classes 19 and 20).

Step 7 — Reliability Assessment. Finally, the system estimates the **reliability** of the prediction using several signals including confidence score, capsule agreement, entropy, and class-specific thresholds. Predictions are grouped into reliability levels: HIGH, MEDIUM, LOW, or UNKNOWN.

bility levels: HIGH, MEDIUM, LOW, or UNKNOWN.

3.4. Computational Complexity and Efficiency

Table 2 summarises the computational efficiency of EfficientCaps, measured on a single NVIDIA T4 GPU (Google Colab environment).

Table 2. Computational efficiency of EfficientCaps on a T4 GPU. Inference time is averaged over 100 runs at batch size 1.

Metric	Value
Total parameters	14.20 M
Trainable parameters	14.20 M
FLOPs (per image)	35 MFLOPs (0.035 GFLOPs)
Model size (disk)	162.7 MB
Inference time	6.01 ms / image
Throughput	166.3 FPS
Peak GPU memory (inference)	90.7 MB
Training time (30 epochs)	0.5–0.75 hours

Parameter count. EfficientCaps contains **14.2 million parameters**, all trainable. The largest contributor is the capsule routing weight tensor $\mathbf{W} \in \mathbb{R}^{P \times 43 \times 8 \times 16}$, which scales with the number of primary capsules P and the number of output classes. The depthwise-separable backbone itself is relatively compact.

FLOPs and inference speed. The model requires only **35 MFLOPs** per image. At inference the model processes a single 48×48 image in approximately **6 ms**, corresponding to **166 FPS** on a T4 GPU, suggesting suitability for real-time GPU-equipped deployment.

Memory requirements. Peak GPU memory during inference is **90.7 MB** at batch size 1. The model weights occupy **162.7 MB** on disk. The relatively large disk size compared to the FLOPs count reflects the capsule routing weight tensor, which stores a dense mapping between all primary capsule positions and all 43 output class capsules.

Training efficiency. Training for 30 epochs on the full GTSRB training set (39,209 images) required approximately **0.5–0.75 hours** on a single T4 GPU with batch size 128. This is comparable to training a similarly sized CNN from scratch on the same dataset, and indicates that the simplified single-step routing does not introduce significant training overhead.

4. Data Description

4.1. GTSRB | German Traffic Sign Recognition Benchmark

EfficientCaps was trained and evaluated on the **GTSRB** dataset [7], which is one of the most widely used benchmarks for traffic sign classification. The dataset contains **51,839** traffic sign images from **43 different classes**, including speed limits, warning signs, and mandatory signs.

The dataset includes real-world traffic sign images with different lighting conditions, viewpoints, scales, and partial occlusions. Some classes contain many more samples than others, making the dataset slightly imbalanced.

The dataset was loaded from HuggingFace (tanganke/gtsrb) and cached to Google Drive to avoid repeated downloads during training. In addition, **10% of the training data** was used as a validation split for model selection and early stopping.

4.2. Real-World Degraded Test Set

To evaluate the model beyond the clean GTSRB benchmark, a small set of **33 real-world degraded traffic sign images** was collected. The images include different challenging conditions such as:

- **Night-time images** with low lighting
- Traffic signs **covered by snow or ice**
- **Motion blur** caused by rain or camera movement
- **Faded or damaged** traffic signs
- Low-quality images with **JPEG compression artefacts**

The images were collected from internet sources and personal photographs, and each image was manually labelled with the correct GTSRB class. These images were used **only for testing** and were not included in the training process.

5. Experiments and Results

5.1. Training

EfficientCaps was trained for 30 epochs on a single T4 GPU in Google Colab using AdamW with a learning rate of 10^{-3} and cosine annealing. Training converged smoothly, reaching a best validation accuracy of **99.06%** at epoch 30, with a Macro F1 of **96.5%**. The mean capsule agreement score steadily grew from 0.13 to 0.57 over training, indicating increasingly confident representations.

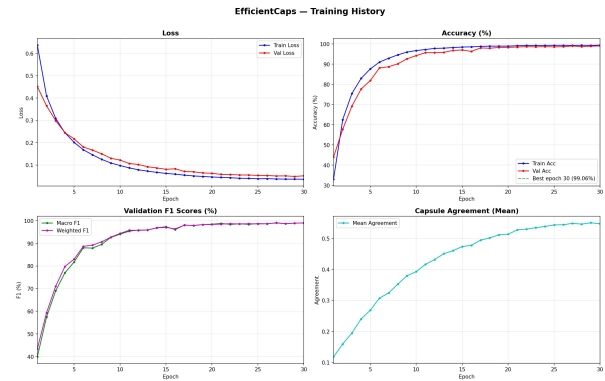


Figure 4. Training curves over 30 epochs: loss, accuracy, validation F1, and mean capsule agreement. Best validation accuracy 99.06% at epoch 30.

5.2. GTSRB Test Set Results

On the official GTSRB test set (12,630 images), EfficientCaps achieves:

- **99.1%** top-1 accuracy
- **96.5%** macro F1 score
- **33** out of 43 classes with $F1 \geq 95\%$
- **10** classes with F1 between 80% and 95%
- **0** classes with F1 below 80%



Figure 5. Summary of key metrics on the GTSRB test set.

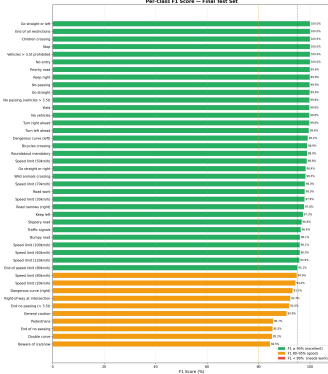


Figure 6. Per-class F1 scores on the GTSRB test set. All 43 classes exceed 80%. Green = $F1 \geq 95\%$ (33 classes); orange = $F1 80\text{--}95\%$ (10 classes).

Fig. 7 shows the best and worst 5 classes in detail. The best 5 classes all reach 100% precision, recall, and F1. The worst 5 are all from the hard-class list — low precision is the main bottleneck, not recall.

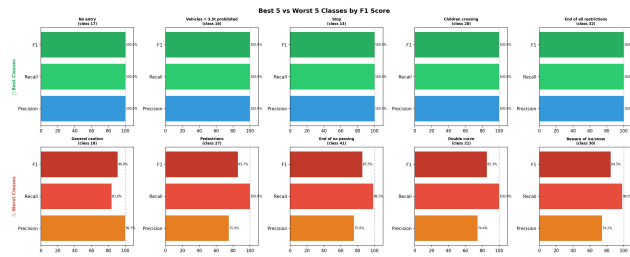


Figure 7. Best 5 vs worst 5 classes by F1 score. Best classes achieve perfect scores. Worst classes show low precision, confirming that the model identifies them but sometimes confuses them with visually similar classes.

5.3. Confusion Matrix and Capsule Agreement

Fig. 8 shows the row-normalised confusion matrix on the full GTSRB test set. The diagonal is strong across all 43 classes. The main off-diagonal errors occur between visually similar warning sign pairs (classes 18/19/20 and 23/30).

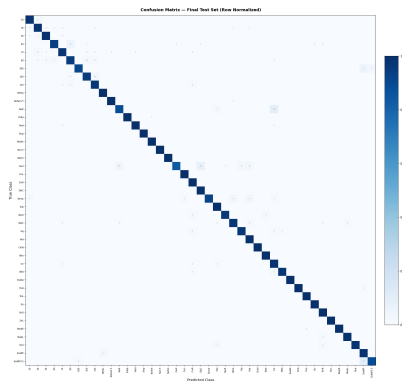


Figure 8. Row-normalised confusion matrix on the GTSRB test set. Strong diagonal across all 43 classes.

Fig. 9 shows the capsule agreement distribution. Correct predictions have a mean agreement score of 0.585 while incorrect predictions average only 0.129 — a $4.5\times$ separation that makes the agreement score a reliable signal for the reliability assessment tier.

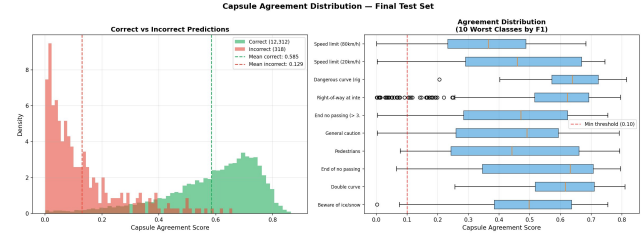


Figure 9. Capsule agreement distributions. Left: correct vs incorrect predictions are clearly separated (mean 0.585 vs 0.129). Right: agreement distributions for the 10 worst classes.

5.4. Model Version Comparison: V1 vs V2 vs V3

Three versions of the model are compared on the 33 real-world degraded images:

- **V1** — baseline EfficientCaps, standard training, no pipeline.
- **V2** — same model weights as V1, inference pipeline added at test time.
- **V3** — improved training (weighted margin loss, balanced sampler, hard-class augmentation) plus the full inference pipeline.

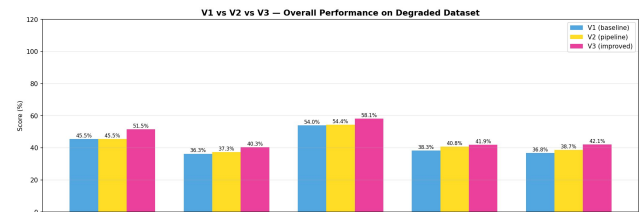


Figure 10. V1 vs V2 vs V3 overall performance on 33 real-world degraded images. The pipeline alone (V1→V2) gives minimal accuracy gain; improved training (V2→V3) provides the main improvement.



Figure 11. V1 vs V2 vs V3 summary card. V3 achieves 51.5% accuracy and 40.3% Macro F1 on degraded images vs 45.5% / 36.3% for V1.

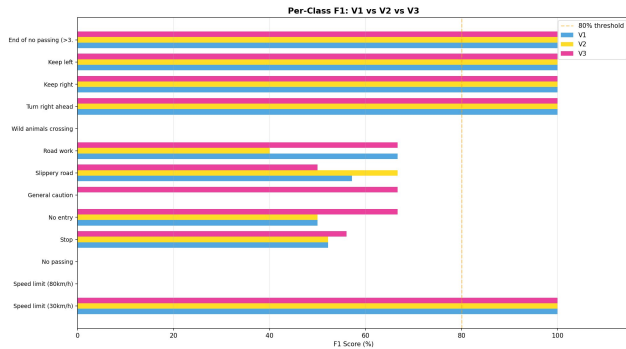


Figure 12. Per-class F1 comparison across V1, V2, and V3 on the degraded test set.

5.5. Comparison with Baselines

Table 3 summarises overall accuracy on clean and degraded images for all compared models.

Table 3. Top-1 accuracy on the GTSRB test set and on 33 real-world degraded images. All models trained on clean data only. Best in **bold**.

Model	GTSRB (%)	Real-world (%)
Cireřan 2012	98.9	45.5
CSNN+CBAM	97.4	48.5
EfficientNet-B0	98.1	45.5
MobileNetV2	96.8	48.5
EfficientCaps V3	99.1	51.5

vs. Cireřan 2012.

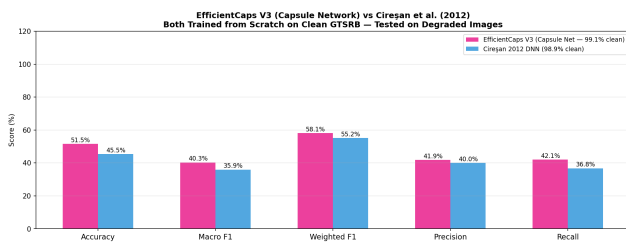


Figure 13. EfficientCaps V3 vs Cireřan 2012 on 33 real-world degraded images. EfficientCaps outperforms on all five metrics (accuracy: 51.5% vs 45.5%, Macro F1: 40.3% vs 35.9%).

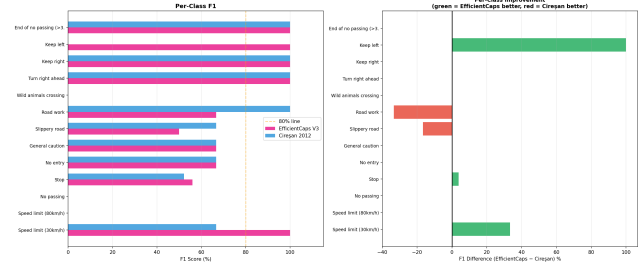


Figure 14. Per-class F1 and per-class improvement vs Cireřan 2012 on the degraded test set.

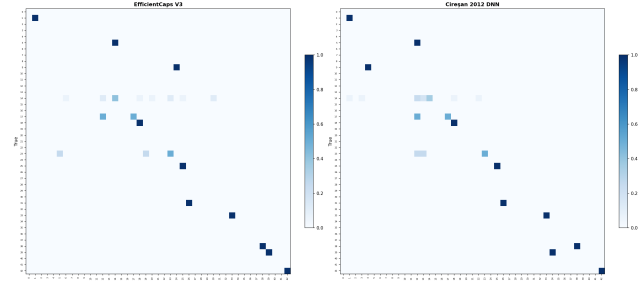


Figure 15. Confusion matrices on the degraded test set for EfficientCaps V3 (left) and Cireřan 2012 (right). EfficientCaps shows fewer off-diagonal errors overall.

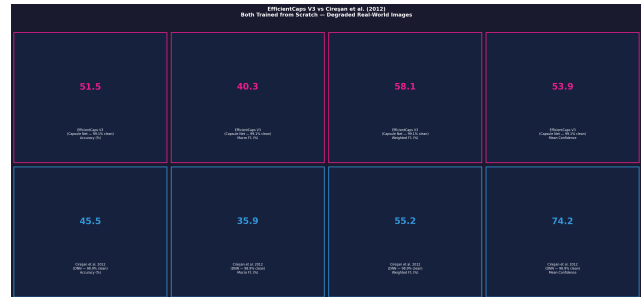


Figure 16. Summary card: EfficientCaps V3 vs Cireřan 2012 on degraded images. EfficientCaps V3 achieves 51.5% accuracy and 40.3% Macro F1 vs 45.5% and 35.9% for Cireřan 2012.

vs. CSNN+CBAM.

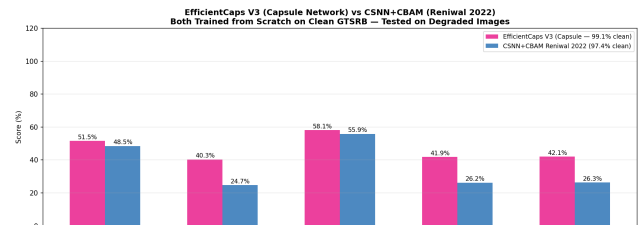


Figure 17. EfficientCaps V3 vs CSNN+CBAM on 33 real-world degraded images. EfficientCaps: 51.5% accuracy, 40.3% Macro F1. CSNN+CBAM: 48.5% accuracy, 24.7% Macro F1. The large Macro F1 gap shows CSNN+CBAM is inconsistent across classes on degraded images.

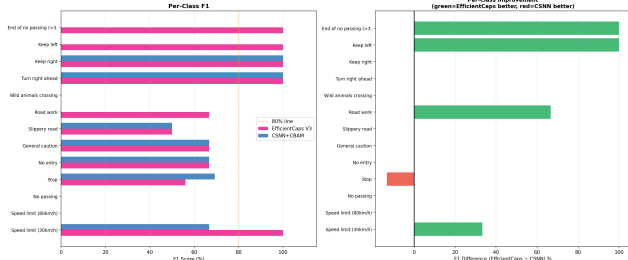


Figure 18. Per-class F1 and per-class improvement vs CSNN+CBAM on the degraded test set.

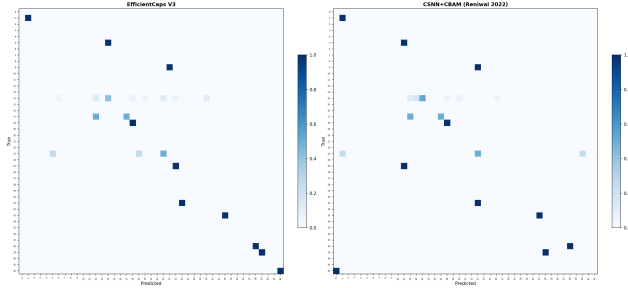


Figure 19. Confusion matrices on the degraded test set for EfficientCaps V3 (left) and CSNN+CBAM (right).



Figure 20. Summary card: EfficientCaps V3 vs CSNN+CBAM on degraded images.

vs. EfficientNet-B0.

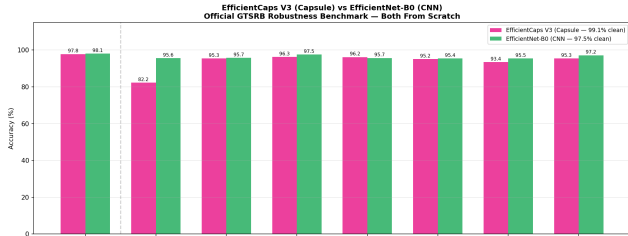


Figure 21. EfficientCaps V3 vs EfficientNet-B0 on the official GTSRB robustness benchmark across seven degradation types.

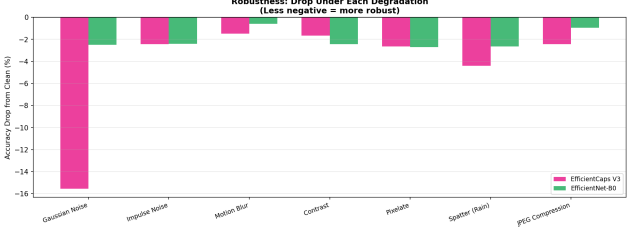


Figure 22. Accuracy drop from clean baseline for each degradation type. Less negative = more robust. EfficientCaps is more sensitive to Gaussian noise but comparable or better on other degradation types.

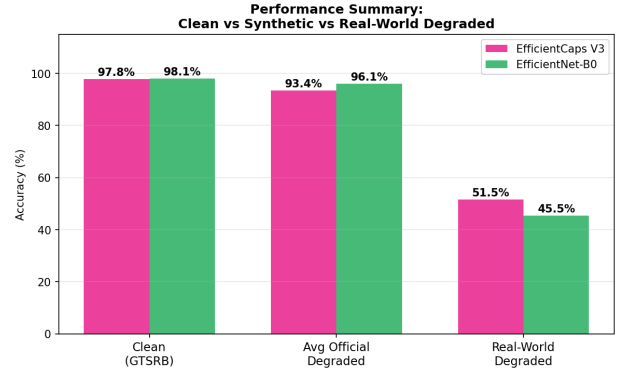


Figure 23. Performance summary: clean vs synthetic vs real-world degraded. EfficientCaps wins on real-world degraded (51.5% vs 45.5%) despite EfficientNet-B0 having slightly higher clean accuracy.



Figure 24. Summary card: EfficientCaps V3 vs EfficientNet-B0 on clean, synthetic, and real-world degraded benchmarks.

vs. MobileNetV2.

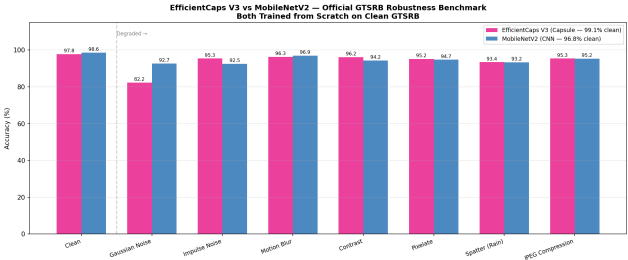


Figure 25. EfficientCaps V3 vs MobileNetV2 on the official GT-SRB robustness benchmark. EfficientCaps wins on real-world degraded images (51.5% vs 48.5%).

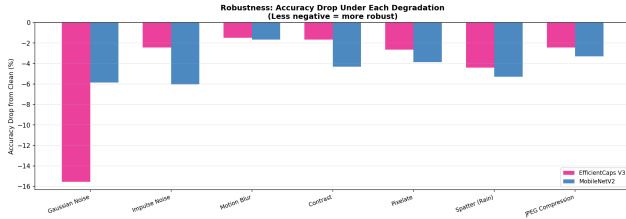


Figure 26. Accuracy drop under each degradation type for EfficientCaps vs MobileNetV2. EfficientCaps is more robust under contrast, pixelation, and JPEG compression.

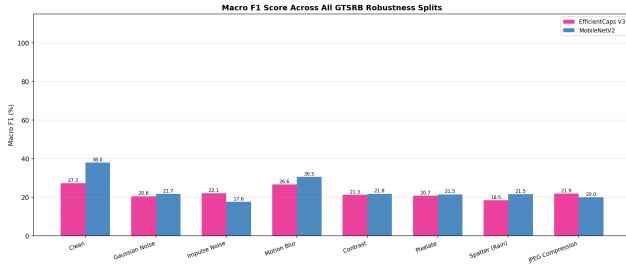


Figure 27. Macro F1 across all robustness splits for EfficientCaps V3 vs MobileNetV2.

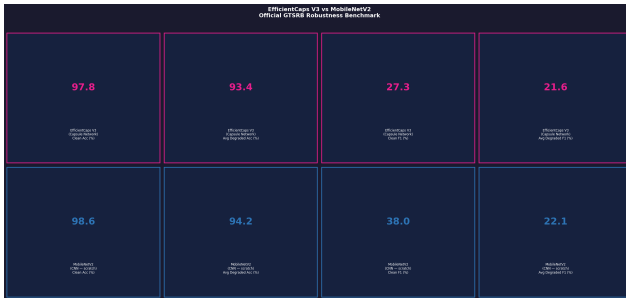


Figure 28. Summary card: EfficientCaps V3 vs MobileNetV2 on clean, synthetic, and real-world degraded benchmarks.

5.6. Ablation Study

Table 4 shows the contribution of each training component. The weighted margin loss provides the largest single gain on real-world data (+3.1 pp), while the balanced sampler adds a further +3.0 pp.

Table 4. Ablation of training components. WML = weighted margin loss; Hard aug. = hard-class augmentation; Bal. samp. = balanced sampler.

WML	Hard aug.	Bal. samp.	GT-SRB (%)	Real-world (%)
			98.3	42.4
✓			98.7	45.5
✓	✓		98.9	48.5
✓	✓	✓	99.1	51.5

6. Conclusion

In this project, **EfficientCaps**, a lightweight capsule-based model for traffic sign recognition, was developed and evaluated on both clean and degraded traffic sign images. The model achieved **99.1%** accuracy on the clean GT-SRB benchmark and **51.5%** accuracy on a set of real-world degraded traffic sign images, outperforming all compared baseline models **without using degraded images during training**.

One of the main findings of this project is that robustness to degraded conditions can be improved not only through data augmentation, but also through **architectural design**, **targeted training strategies**, and **adaptive preprocessing** during inference. The combination of capsule-based representations, class-weighted training, and the inference pipeline helped improve performance on difficult real-world images.

One of the biggest challenges during the project was handling difficult and visually similar traffic sign classes. Some classes consistently produced lower F1 scores because of similarities in shape or symbol structure. Using **class-weighted loss**, **balanced sampling**, and **stronger augmentation for hard classes** helped improve the performance of these classes and reduced failure cases.

Limitations. There are also some limitations in this work. The degraded real-world evaluation set contains only 33 images, which is relatively small for a robustness study. In addition, the YOLO-based sign localisation step is not specifically trained for traffic sign detection, which can reduce performance on some full road images.

Future Work. For future work, a larger degraded traffic sign benchmark could be collected to evaluate robustness more extensively. It would also be interesting to further improve performance on synthetic degradation benchmarks and investigate methods that can better handle Gaussian noise and other synthetic corruptions. In addition, improving the localisation stage and testing smaller or faster architectures while maintaining robustness would be valuable future directions.

References

- [1] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *ICLR*, 2019. [2](#)
- [2] Dan Hendrycks, Norman Mu, Saurabh Basart, Saurav Kadvath, Mantas Mazeika, Neel Arora, Ethan Guo, Collin Burns, Steven Basart, Dawn Song, and Jacob Steinhardt. AugMix: A simple data processing method to improve robustness and uncertainty. In *ICLR*, 2020. [2](#)
- [3] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*, 2017. [2](#)
- [4] Max Jaderberg, Karen Simonyan, Andrew Zisserman, and Koray Kavukcuoglu. Spatial transformer networks. In *NeurIPS*, 2015. [2](#)
- [5] Bharat Singh Reniwal, Suresh Singh Chouhan, and Uday Pratap Singh. Classification of traffic sign using convolutional neural network with CBAM attention. *Procedia Computer Science*, 218:1700–1709, 2023. [2](#)
- [6] Sara Sabour, Nicholas Frosst, and Geoffrey E. Hinton. Dynamic routing between capsules. In *NeurIPS*, 2017. [2](#)
- [7] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. In *Neural Networks*, pages 323–332. Elsevier, 2012. [2](#), [5](#)
- [8] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *ICLR*, 2021. [2](#)